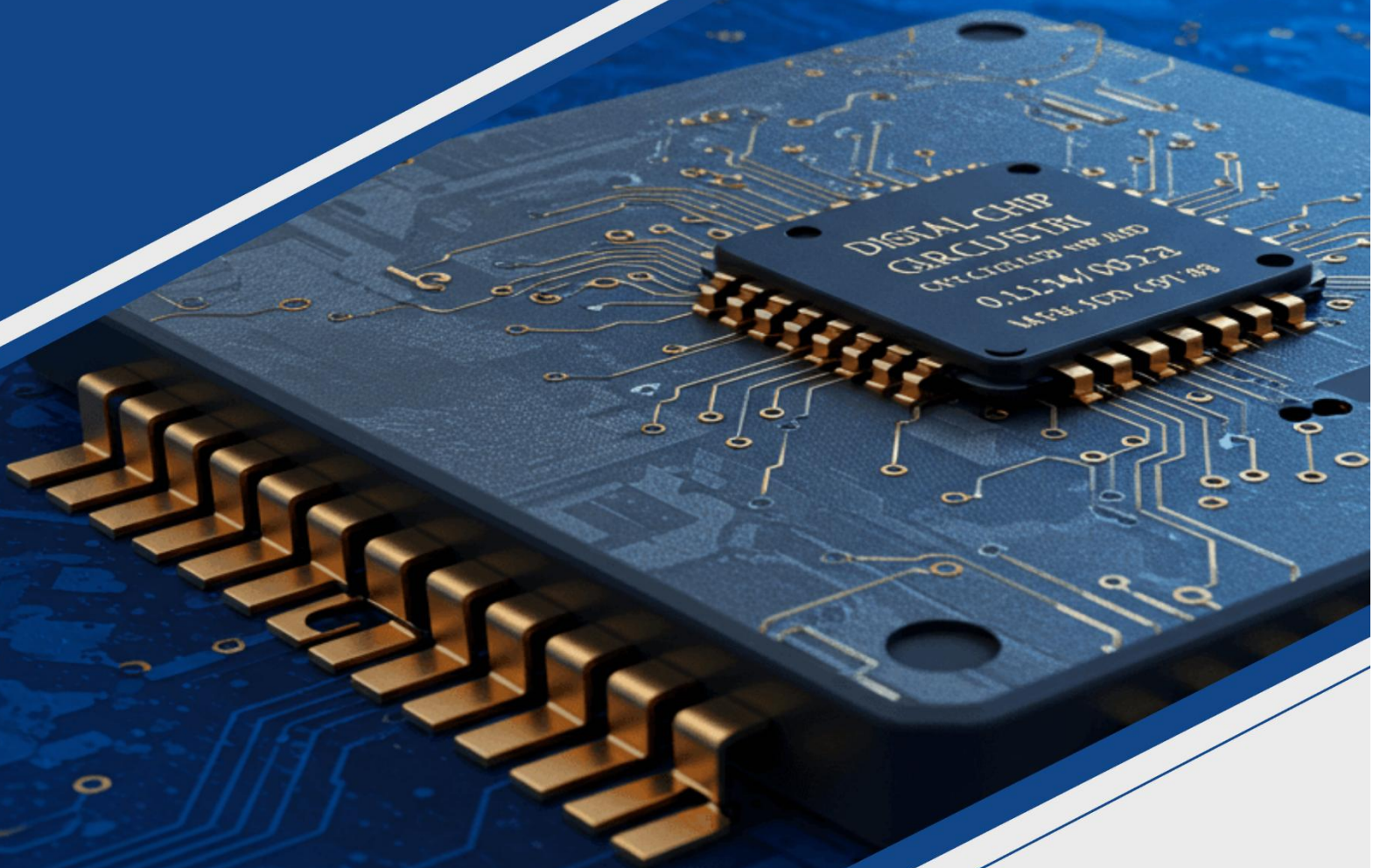




Temon & Bomba Team

Digital Design

SPI Project



Provided by
Saeed Nabawy

Zeyad Abdallah Shaban
Mohamed Hassan Abdelkhalek
Submitted to
ENG / Kareem Waseem

Table of Contents

Introduction	5
Project Description.....	6
Overview	6
Top-Level Module: SPI_Wrapper	6
SPI_Slave Module	7
RAM Module.....	7
Data Conversion Modules.....	7
Serial_to_Parallel	7
parallel_to_serial	7
SPI_Wrapper_tb (Testbench)	8
Modules Code	9
Serial_to_Parallel Module	9
Parallel_to_Serial Module	10
RAM Module.....	11
SPI_Slave Module	12
SPI_Wrapper Module.....	14
Testbench.....	15
Do File	16
QuestaSim Simulation.....	17
Write Address.....	17
Write Data.....	17
Data Modified in Memory	18
Read Address	18
Data in Memory Address 'h255 ('d85)	18
Read Data & MISO output	19
Linting	19
Elaboration (seq encoding)	20
Message.....	20

Schematic.....	20
RAM Schematic	21
SPI Schematic	21
Synthesis (seq encoding)	22
Schematic.....	22
SPI_Slave Schematic.....	23
Message.....	24
Utilization Report.....	24
Timing Report.....	24
Critical path highlighted in the schematic	25
Implementation (seq encoding)	26
Device.....	26
Message.....	27
Utilization Report.....	27
Timing Report.....	27
Constrain File	28
Elaboration (gray encoding)	29
Message.....	29
Schematic.....	29
Synthesis (gray encoding)	30
Schematic.....	30
SPI_Slave Schematic.....	31
Message.....	32
Utilization Report.....	32
Timing Report.....	32
Implementation (gray encoding)	33
Device.....	33
Message.....	34
Utilization Report.....	34

Timing Report.....	34
Elaboration (one_hot encoding)	35
Message.....	35
Schematic.....	35
Synthesis (one_hot encoding)	36
Schematic.....	36
SPI_Slave Schematic.....	37
Message.....	38
Utilization Report.....	38
Timing Report.....	38
Implementation (one_hot encoding)	39
Device.....	39
Message.....	40
Utilization Report.....	40
Timing Report.....	40

Introduction

In the ever-evolving world of embedded systems, devices must speak a common language — one not built on human words, but on precise timing, pulses, and codes. Deep within the silicon silence of microcontrollers and peripheral chips, the Serial Peripheral Interface (SPI) is that language: a minimalist protocol, sharp and efficient, whispering commands and responses in synchronous rhythm.

This project delves into the hidden patterns of this digital dialogue. We design and implement a custom SPI Slave system in Verilog — a silent listener that not only interprets incoming serial data but acts on it intelligently. Like a watchful gatekeeper, it receives instructions, deciphers intent, and manipulates memory with precision.

The design merges state-machine logic with serial-to-parallel and parallel-to-serial data conversion. It features an internal RAM array, capable of storing and retrieving data based on encoded commands, all processed in real time through carefully synchronized hardware logic.

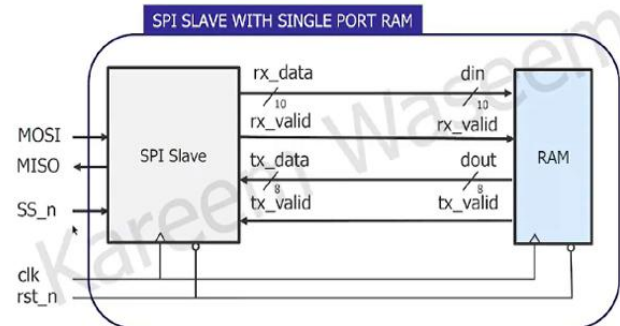
What begins as a stream of shifting bits over a single wire becomes, in this system, a fully responsive, memory-aware peripheral — crafted not just to understand the SPI protocol, but to embody it.

Project Description

Overview

This project implements a **complete SPI Slave system in Verilog**, with a built-in memory subsystem and full support for read/write communication initiated by an SPI Master. It includes all necessary components for:

- Serial-to-parallel conversion (to interpret incoming SPI data),
- Parallel-to-serial conversion (to send responses back),
- Command decoding,
- Addressable memory storage (RAM),
- And a testbench to simulate real-world usage.



Top-Level Module: SPI_Wrapper

This module acts as the central unit. It wires together:

- The SPI_Slave module, which handles protocol logic.
- The RAM module, which stores and retrieves data based on SPI commands.

Ports:

- Inputs: MOSI, SS_n, clk, rst_n
- Output: MISO

Internal Signals:

- rx_data [9:0] – full 10-bit command/data received from SPI
- rx_valid – signals when a valid 10-bit word has been received
- tx_data [7:0] – data to be transmitted over SPI
- tx_valid – signals when data is ready to be transmitted

SPI_Slave Module

This is the protocol engine. It:

- Uses an FSM with five states: IDLE, CHK_CMD, WRITE, READ_ADD, READ_DATA
- Receives 10-bit commands using the Serial_to_Parallel module
- Transmits 8-bit data using the parallel_to_serial module
- Signals data readiness with rx_valid, and waits for tx_valid to begin transmission

RAM Module

A simple 256-byte memory (8-bit wide, 8-bit addressable), supporting:

- Address selection via SPI
- Data storage and retrieval
- Behavioral simulation support via \$readmemh

It interprets the rx_data coming from the slave:

- 00 opcode sets the internal write address
- 01 opcode writes data to that address
- 10 opcode sets the read address
- 11 opcode reads data from that address and outputs it on dout

Data Conversion Modules

Serial_to_Parallel

- Converts incoming SPI stream into 10-bit parallel words
- Detects transaction completion and triggers done pulse

parallel_to_serial

- Converts 8-bit parallel data to serial format
- Manages busy signal to indicate transmission state

SPI_Wrapper_tb (Testbench)

The testbench simulates:

- Clock and reset behavior
- Command sequences for write and read operations
- Realistic SPI timing (bit-wise shifting over MOSI)
- Memory preload using `$readmemh("mem.dat", ...)`

Test Phases:

1. Reset system
2. Write address
3. Write data
4. Read address
5. Read data
6. Wait for response, verify behavior

Modules Code

Serial_to_Parallel Module



```
1  module Serial_to_Parallel(  
2      input clk,  
3      input rst_n,  
4      input serial_in,  
5      input enable,  
6      output reg [9:0] parallel_out,  
7      output reg done  
8  );  
9  
10  
11  
12      reg [3:0] bit_count;  
13      reg [9:0] shift_reg;  
14  
15      always @(posedge clk) begin  
16          if (!rst_n) begin  
17              shift_reg    <= 10'b0;  
18              bit_count    <= 4'b0;  
19              done         <= 1'b0;  
20              parallel_out <= 10'b0;  
21          end  
22          else if (enable) begin  
23              shift_reg <= {shift_reg[8:0], serial_in};  
24              bit_count <= bit_count + 1;  
25  
26              if (bit_count == 4'b1001) begin  
27                  parallel_out <= {shift_reg[8:0], serial_in};  
28                  shift_reg <= 10'b0;  
29                  done <= 1'b1;  
30                  bit_count <= 4'b0;  
31              end else begin  
32                  done <= 1'b0;  
33              end  
34          end else begin  
35              done <= 1'b0;  
36          end  
37      end  
38  
39  endmodule  
40
```

Parallel_to_Serial Module

```
1  module parallel_to_serial (
2      input wire clk,
3      input wire rst_n,
4      input wire [7:0] parallel_in,
5      input wire load,
6      output reg serial_out,
7      output reg busy
8  );
9
10     reg [7:0] shift_reg;
11     reg [2:0] bit_count;
12
13     always @(posedge clk) begin
14         if (!rst_n) begin
15             shift_reg <= 8'd0;
16             bit_count <= 3'd0;
17             serial_out <= 1'bz;
18             busy <= 1'b0;
19         end else if (load && !busy) begin
20             shift_reg <= parallel_in;
21             bit_count <= 3'd0;
22             busy <= 1'b1;
23         end
24         else if (busy) begin
25             serial_out <= shift_reg[7];
26             shift_reg <= {shift_reg[6:0], 1'b0};
27             bit_count <= bit_count + 1;
28
29             if (bit_count == 3'd7) begin
30                 busy <= 1'b0;
31             end
32         end
33         else begin
34             serial_out <= 1'bz;
35             busy <= 1'b0;
36         end
37     end
38
39 endmodule
40
```

RAM Module

```
1  module RAM (  
2      input [9:0] din,  
3      input rx_valid,clk,rst_n,  
4      output reg tx_valid,  
5      output reg [7:0] dout  
6  );  
7  
8      parameter MEM_DEPTH = 256;  
9      parameter ADDR_SIZE = 8;  
10  
11     reg [ADDR_SIZE-1:0] write_address;  
12     reg [ADDR_SIZE-1:0] read_address;  
13  
14     reg [7:0] mem [0:MEM_DEPTH-1];  
15  
16  
17     always @(posedge clk) begin  
18         if(!rst_n) begin  
19             dout <= 0;  
20             tx_valid <= 0;  
21         end  
22         else begin  
23             if(rx_valid) begin  
24                 case(din[9:8])  
25                     2'b00 : begin  
26                         write_address <= din[7:0];  
27                         dout <= 8'bzzzzzzzz;  
28                         tx_valid <= 1'bz;  
29                     end  
30                     2'b01 : begin  
31                         mem [write_address] <= din[7:0];  
32                         dout <= 8'bzzzzzzzz;  
33                         tx_valid <= 1'bz;  
34                     end  
35                     2'b10 : begin  
36                         read_address <= din[7:0];  
37                         dout <= 8'bzzzzzzzz;  
38                         tx_valid <= 1'bz;  
39                     end  
40                     2'b11 : begin  
41                         dout <= mem [read_address];  
42                         tx_valid <= 1;  
43                     end  
44                     default : begin  
45                         dout <= 8'bzzzzzzzz;  
46                         tx_valid <= 1'bz;  
47                     end  
48                 endcase  
49             end  
50             else begin  
51                 dout <= 0;  
52                 tx_valid <= 0;  
53             end  
54         end  
55     end  
56  
57 endmodule //RAM
```

SPI_Slave Module

```
1  module SPI_Slave(  
2      input MOSI,  
3      input SS_n,  
4      input clk,  
5      input rst_n,  
6      input [7:0] tx_data,  
7      input tx_valid,  
8      output MISO,  
9      output reg [9:0] rx_data,  
10     output reg rx_valid  
11 );  
12  
13     parameter IDLE = 5'b000;  
14     parameter CHK_CMD = 5'b001;  
15     parameter WRITE = 5'b010;  
16     parameter READ_ADD = 5'b011;  
17     parameter READ_DATA = 5'b100;  
18  
19     // (* fsm_encoding = "gray" *)  
20     reg [4:0] cs_ns;  
21     reg addr_or_data ;  
22     reg STP_enable;  
23     wire STP_Done;  
24     wire [9:0] STP_out;  
25     wire PTS_busy, PTS_out;  
26  
27     Serial_to_Parallel STP (.clk(clk), .rst_n(rst_n), .serial_in(MOSI),  
28     .enable(STP_enable), .parallel_out(STP_out), .done(STP_Done));  
29  
30     parallel_to_serial PTS (.clk(clk), .rst_n(rst_n), .parallel_in(tx_data),  
31     .load(tx_valid), .serial_out_buff(MISO), .busy(PTS_busy));  
32  
33     always @(cs,SS_n) begin  
34         case(cs)  
35             IDLE: begin  
36                 if(SS_n) begin  
37                     ns = IDLE;  
38                 end  
39                 else begin  
40                     ns = CHK_CMD;  
41                 end  
42             end  
43             CHK_CMD: begin  
44                 if(SS_n) begin  
45                     ns = IDLE;  
46                 end  
47                 else begin  
48                     if(MOSI) begin  
49                         if(addr_or_data) begin  
50                             ns = READ_DATA;  
51                         end  
52                         else begin  
53                             ns = READ_ADD;  
54                         end  
55                     end  
56                     else begin  
57                         ns = WRITE;  
58                     end  
59                 end  
60             end  
61             WRITE: begin  
62                 if(SS_n) begin  
63                     ns = IDLE;  
64                 end  
65                 else begin  
66                     ns = WRITE;  
67                 end  
68             end  
69             READ_ADD: begin  
70                 if(SS_n) begin  
71                     ns = IDLE;  
72                 end  
73                 else begin  
74                     ns = READ_ADD;  
75                 end  
76             end  
77             READ_DATA: begin  
78                 if(SS_n) begin  
79                     ns = IDLE;  
80                 end  
81                 else begin  
82                     ns = READ_DATA;  
83                 end  
84             end  
85             default : ns = IDLE;  
86         endcase  
87     end  
88
```



```
1      always @(posedge clk or negedge rst_n) begin
2          if(!rst_n) begin
3              cs <= IDLE;
4          end
5          else begin
6              cs <= ns;
7          end
8      end
9
10     always @(posedge clk) begin
11         if(cs == IDLE || cs == CHK_CMD) begin
12             STP_enable <= 0;
13             rx_data<= {10{1'bz}};
14             rx_valid <= 1'b0;
15         end
16         else if(cs == WRITE) begin
17             STP_enable <= 1;
18             if(STP_Done) begin
19                 rx_valid <= 1;
20                 rx_data <= STP_out;
21                 STP_enable <= 0;
22             end
23         end
24         else if(cs == READ_ADD) begin
25             STP_enable <= 1;
26             if(STP_Done) begin
27                 rx_valid <= 1;
28                 rx_data <= STP_out;
29                 addr_or_data <= 1;
30                 STP_enable <= 0;
31             end
32         end
33         else if(cs == READ_DATA) begin
34             STP_enable <= 1;
35             if(STP_Done) begin
36                 rx_valid <= 1;
37                 rx_data<= STP_out;
38                 addr_or_data <= 0;
39                 STP_enable <= 0;
40             end
41         end
42     end
43 endmodule
```

SPI_Wrapper Module



```
1  module SPI_Wrapper(  
2      input MOSI,  
3      input SS_n,  
4      input clk,  
5      input rst_n,  
6      output MISO  
7  );  
8      wire [9:0] rx_data;  
9      wire rx_valid;  
10     wire [7:0] tx_data;  
11     wire tx_valid;  
12  
13     SPI_Slave SPI (.MOSI(MOSI),.SS_n(SS_n),.clk(clk),.rst_n(rst_n),  
14         .MISO(MISO),.rx_data(rx_data),.rx_valid(rx_valid),.tx_data(tx_data),  
15         .tx_valid(tx_valid));  
16  
17     RAM u_ram(.din(rx_data),.rx_valid(rx_valid),.dout(tx_data),  
18         .tx_valid(tx_valid),.clk(clk),.rst_n(rst_n));  
19  
20 endmodule
```

Testbench

```
1  module SPI_Wrapper_tb();
2
3      reg MOSI,SS_n,clk,rst_n;
4      wire MISO;
5
6      SPI_Wrapper SPI(.MOSI(MOSI),.SS_n(SS_n),.clk(clk),
7                      .rst_n(rst_n),.MISO(MISO));
8
9      initial begin
10         clk = 0;
11         forever #1 clk = ~clk;
12     end
13
14     integer i;
15     initial begin
16         $readmemh("mem.dat", SPI.u_ram.mem);
17         // resq test
18         rst_n = 0;
19         @(posedge clk);
20         rst_n = 1;
21
22         // Write Address
23         @(posedge clk);
24         SS_n = 0;
25         @(posedge clk);
26         MOSI = 0;
27         @(posedge clk);
28         for(i = 0; i < 10; i = i + 1) begin
29             if(i < 2) begin
30                 MOSI = 0;
31             end
32             else begin
33                 MOSI = ~MOSI;
34             end
35             @(posedge clk);
36         end
37         SS_n = 1;
38
39         // Write Data
40         @(posedge clk);
41         SS_n = 0;
42         @(posedge clk);
43         MOSI = 0;
44         @(posedge clk);
45         for(i = 0; i < 10; i = i + 1) begin
46             if(i == 0) begin
47                 MOSI = 0;
48             end
49             else begin
50                 MOSI = 1;
51             end
52             @(posedge clk);
53         end
54         SS_n = 1;
55
56         // Read Address
57         @(posedge clk);
58         SS_n = 0;
59         @(posedge clk);
60         MOSI = 1;
61         @(posedge clk);
62         for(i = 0; i < 10; i = i + 1) begin
63             if(i == 0) begin
64                 MOSI = 1;
65             end
66             else if(i == 1) begin
67                 MOSI = 0;
68             end
69             else begin
70                 MOSI = ~MOSI;
71             end
72             @(posedge clk);
73         end
74         SS_n = 1;
75
76         // Read Data
77         @(posedge clk);
78         SS_n = 0;
79         @(posedge clk);
80         MOSI = 1;
81         @(posedge clk);
82         for(i = 0; i < 10; i = i + 1) begin
83             MOSI = 1;
84             @(posedge clk);
85         end
86         for(i = 0; i < 20; i = i + 1) begin
87             @(posedge clk);
88         end
89         SS_n = 1;
90         $stop;
91     end
92 end
93 endmodule
```

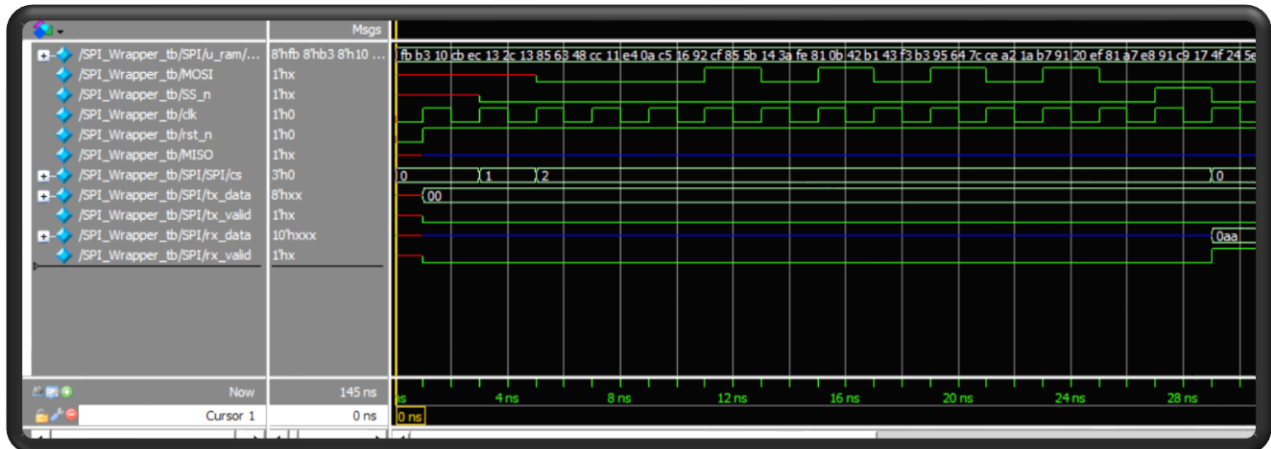
Do File



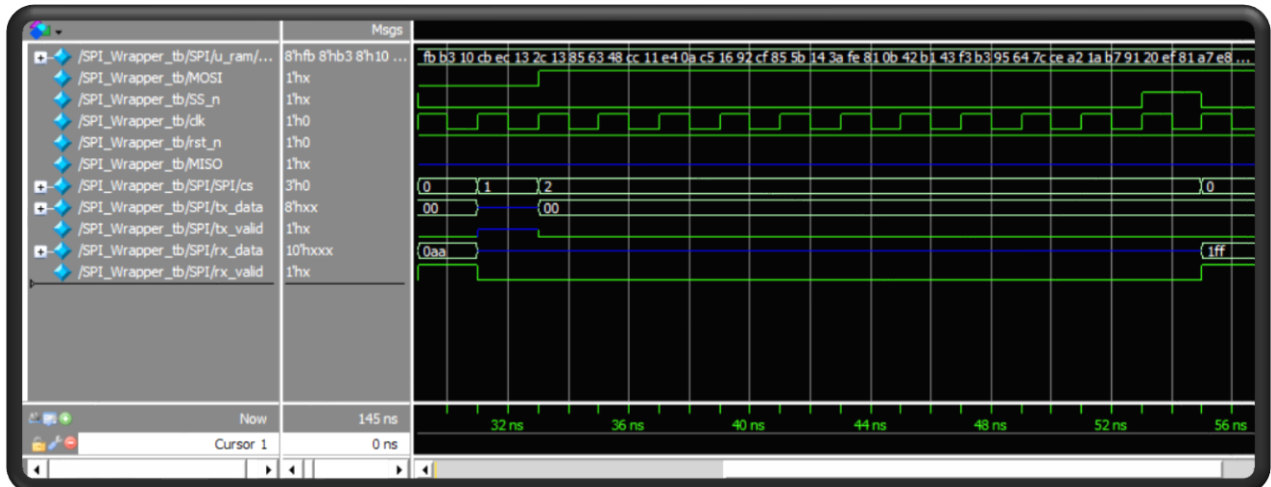
```
1  vlib work
2  vlog RAM.v Serial_to_Parallel.v Parallel_to_Serial.v SPI_Slave.v SPI_Wrapper.v
3  vsim -voptargs=+acc work.SPI_Wrapper_tb
4  add wave -position insertpoint sim:/SPI_Wrapper_tb/*
5  add wave -position insertpoint \
6  sim:/SPI_Wrapper_tb/SPI/rx_data \
7  sim:/SPI_Wrapper_tb/SPI/rx_valid \
8  sim:/SPI_Wrapper_tb/SPI/tx_data \
9  sim:/SPI_Wrapper_tb/SPI/tx_valid
10 add wave -position insertpoint \
11 sim:/SPI_Wrapper_tb/SPI/SPI/cs
12 add wave -position 0  sim:/SPI_Wrapper_tb/SPI/u_ram/mem
13 run -all
14 #quit -sim
```

QuestaSim Simulation

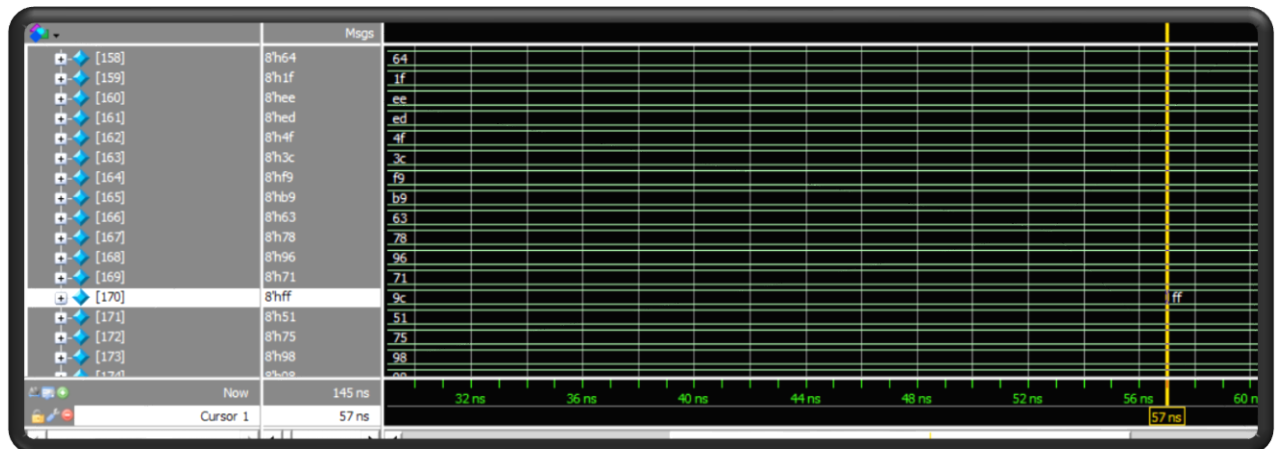
Write Address



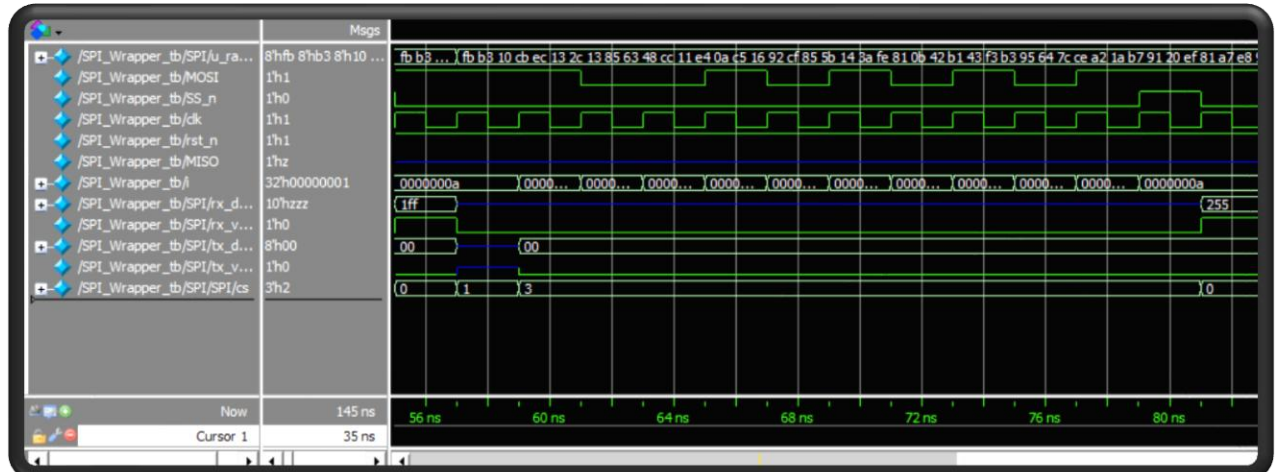
Write Data



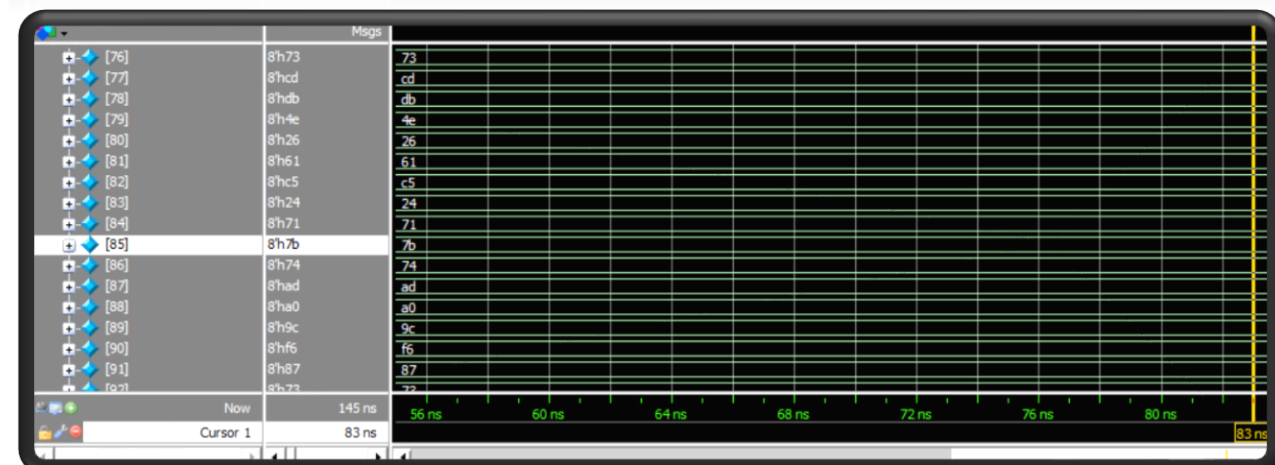
Data Modified in Memory



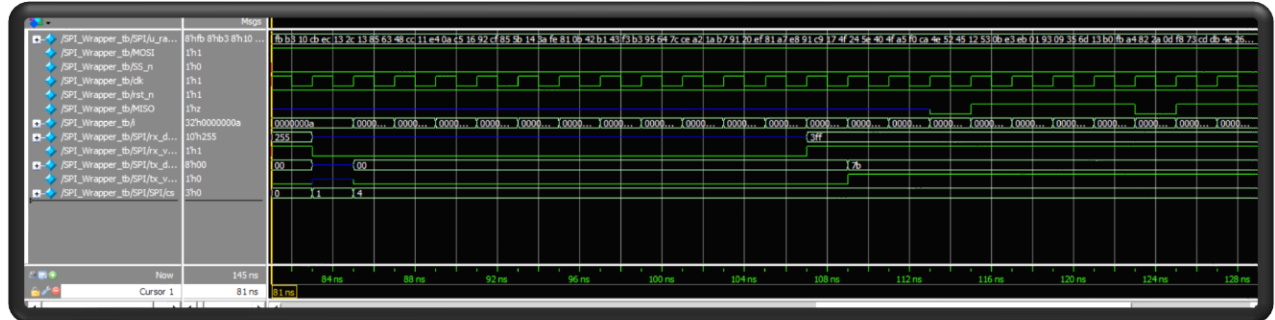
Read Address



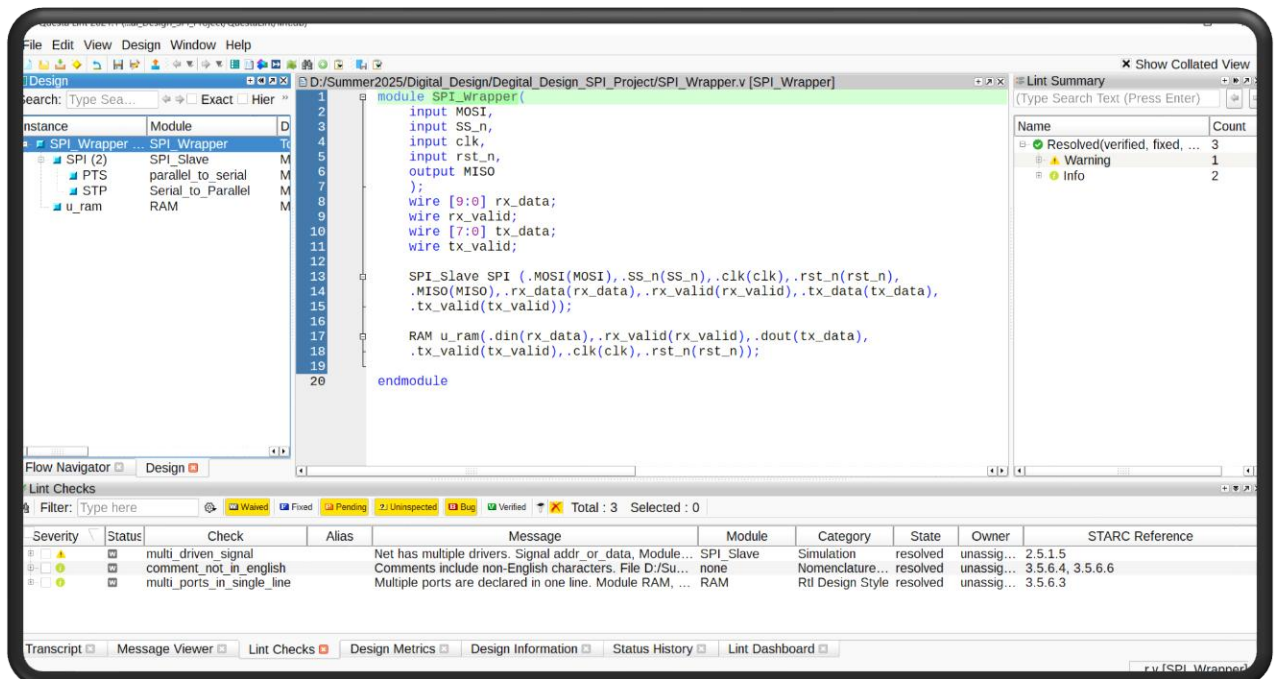
Data in Memory Address 'h255 ('d85)



Read Data & MISO output

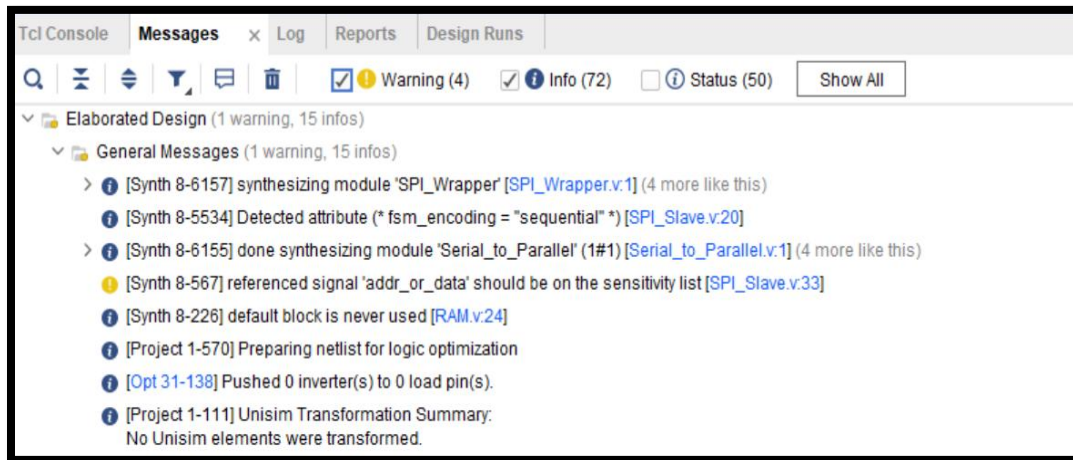


Linting

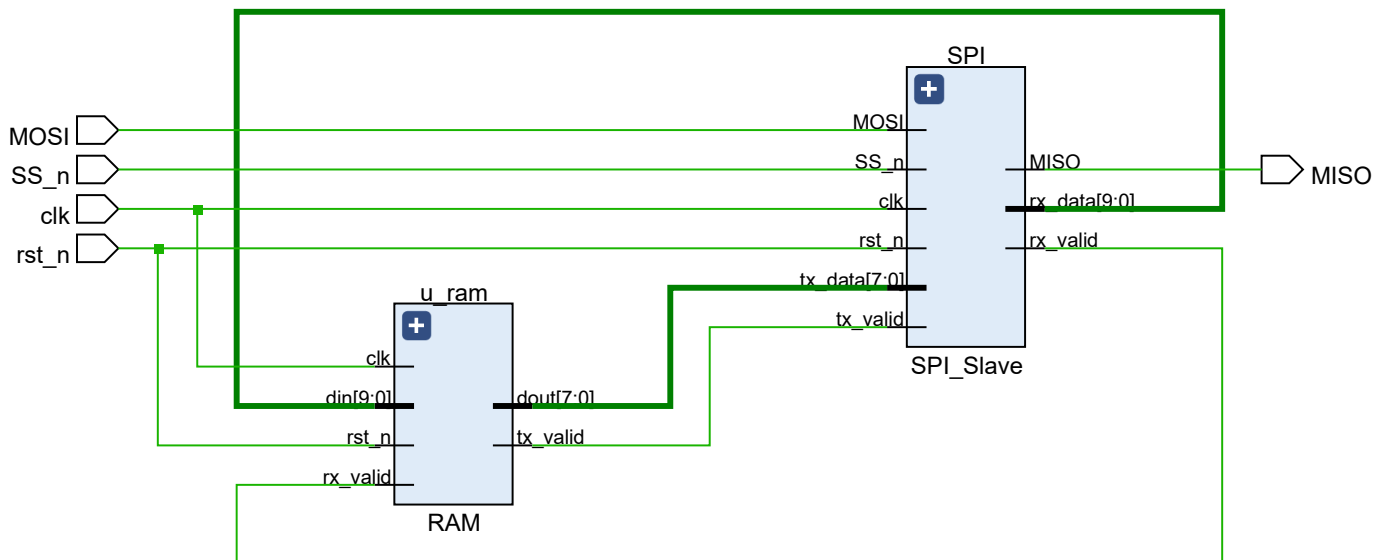


Elaboration (seq encoding)

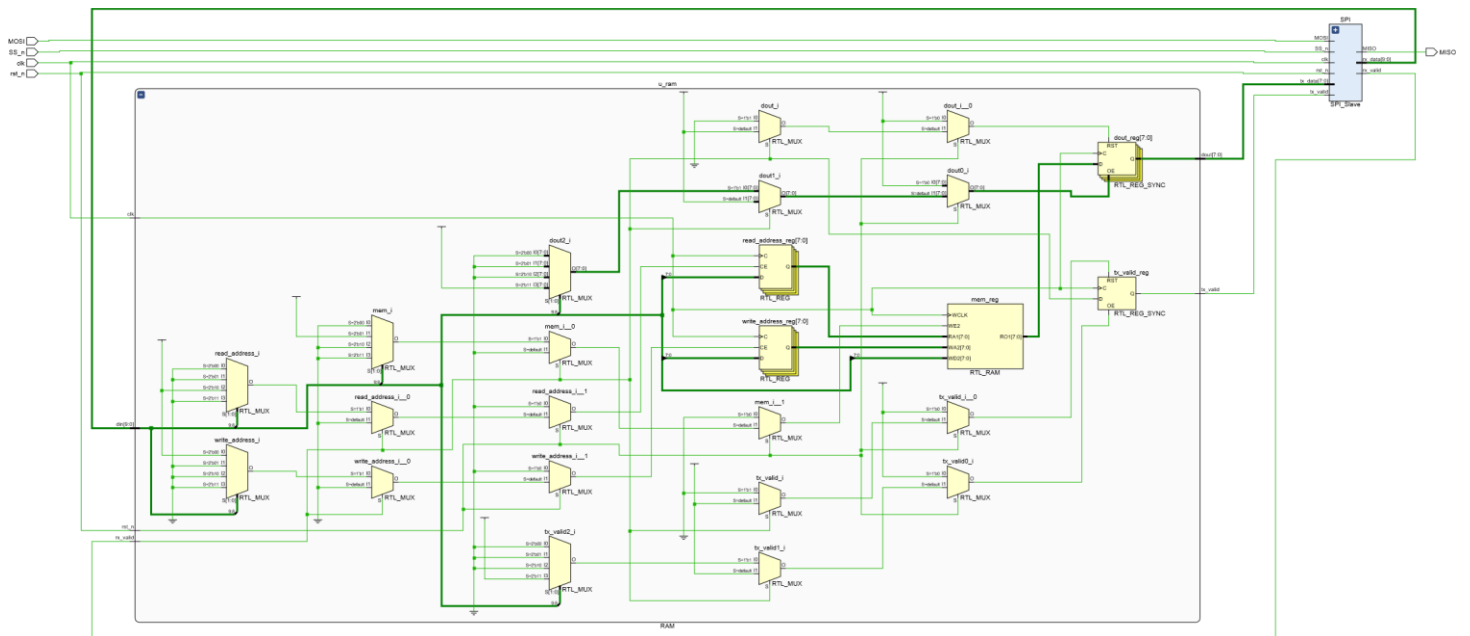
Message



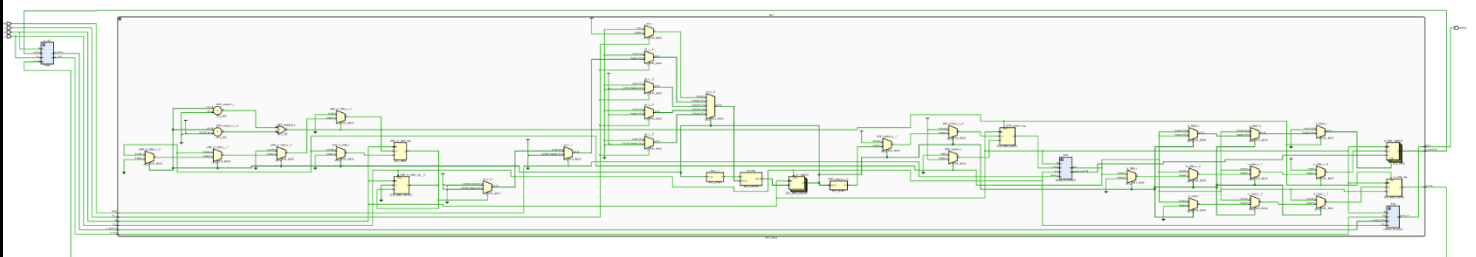
Schematic



RAM Schematic



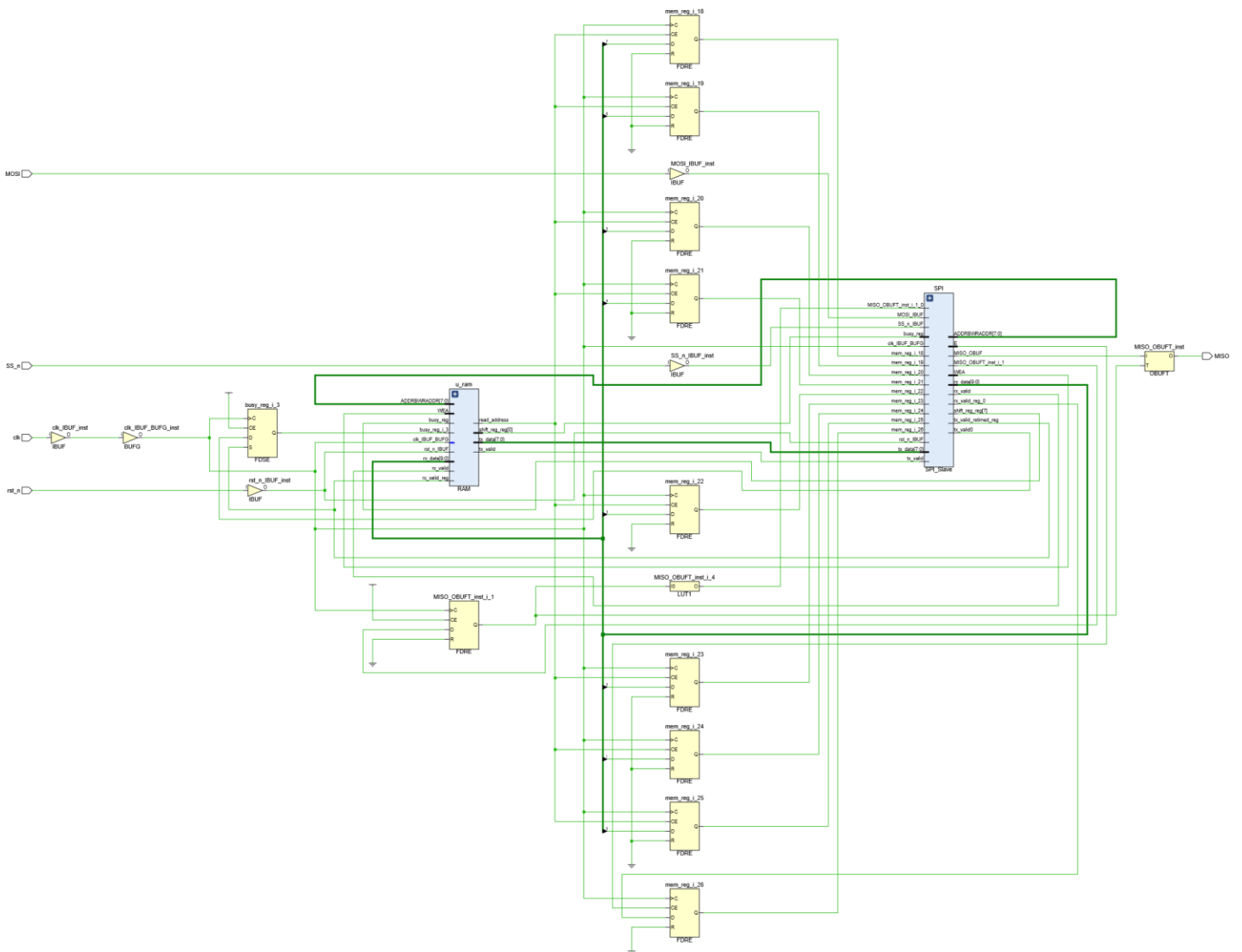
SPI Schematic



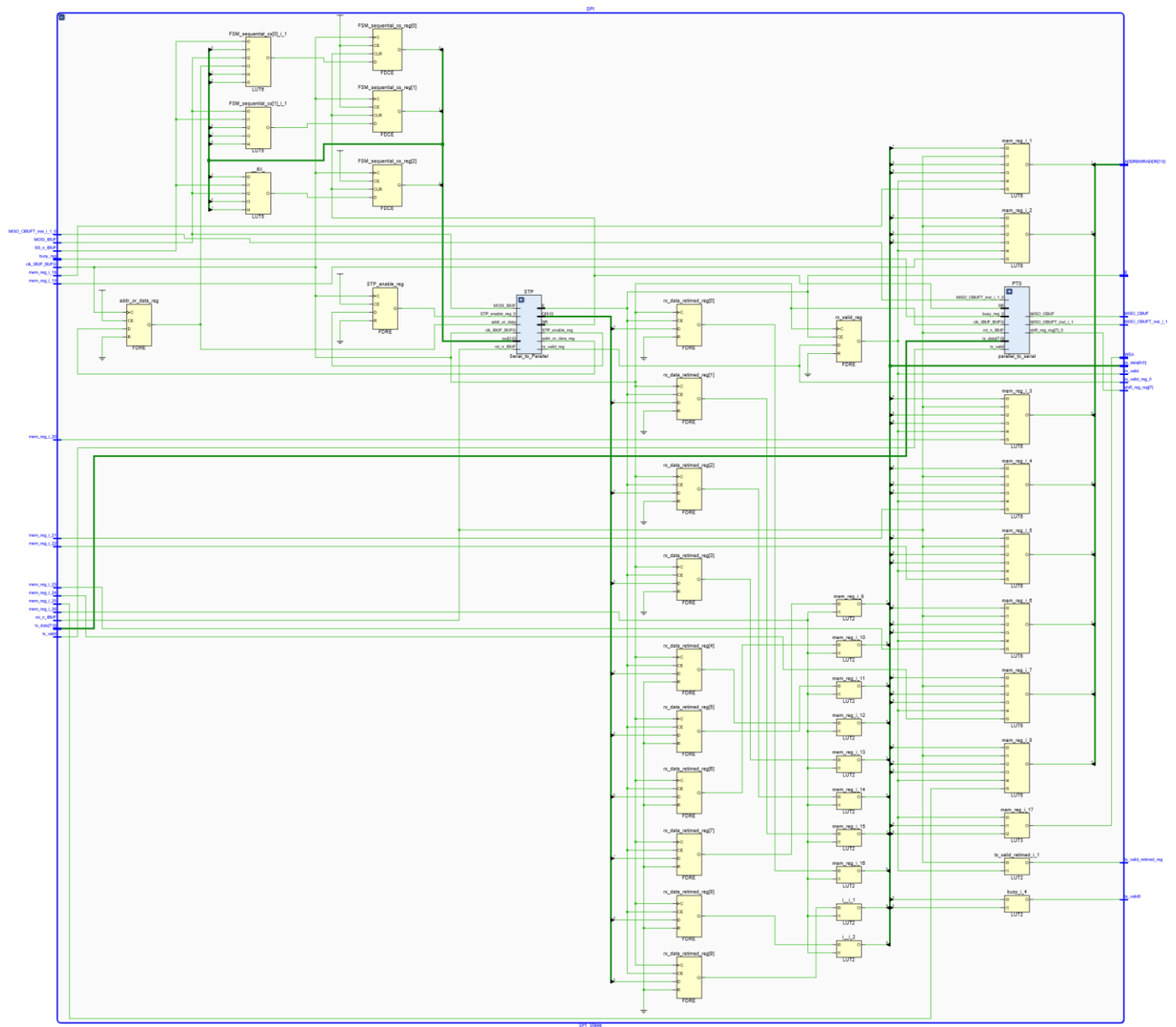
Synthesis (seq encoding)

State	Encoding
IDLE	5'b000
CHK_CMD	5'b001
WRITE	5'b010
READ_ADD	5'b011
READ_DATA	5'b100

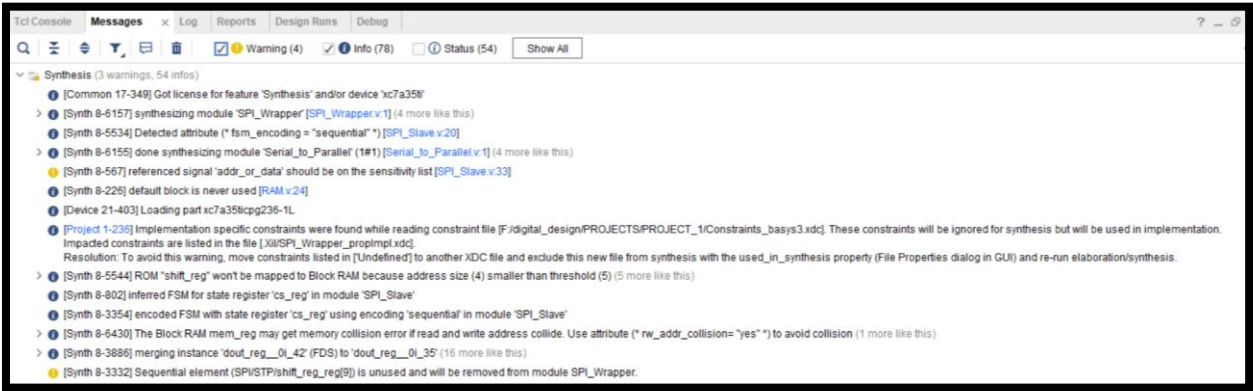
Schematic



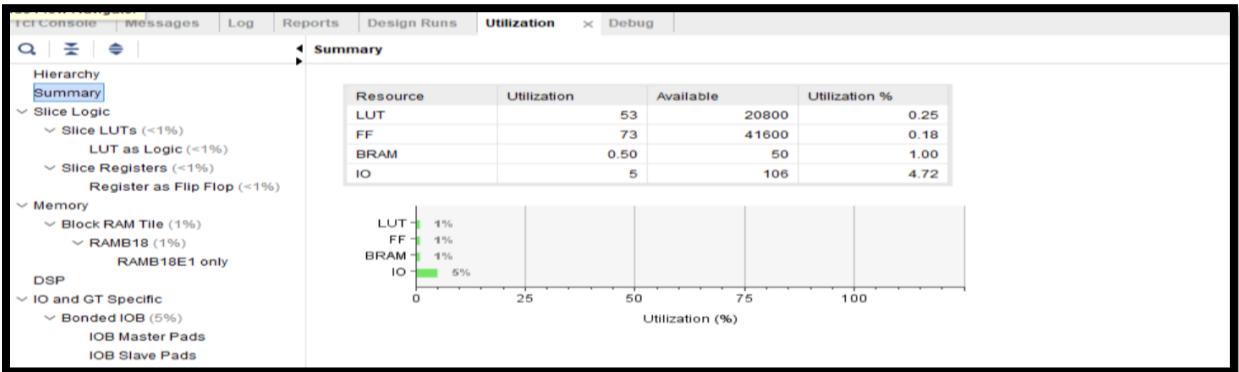
SPI_Slave Schematic



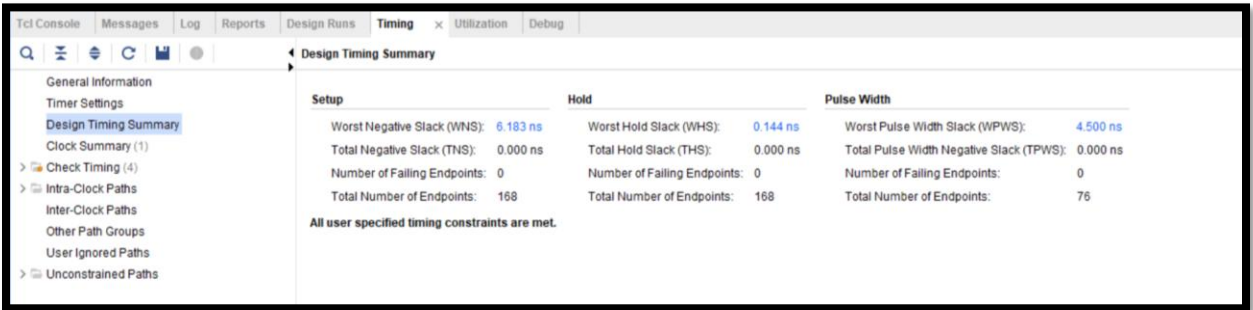
Message



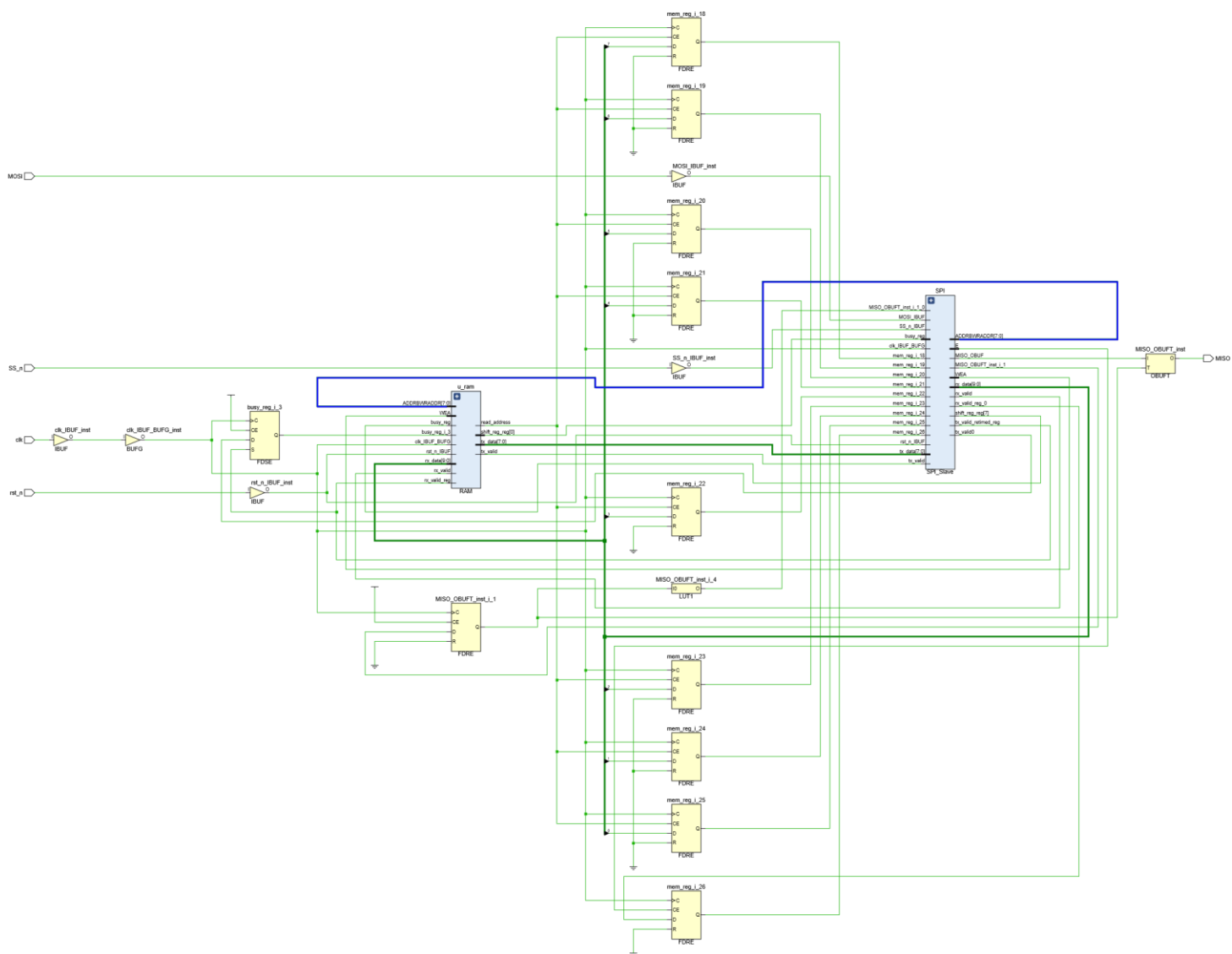
Utilization Report



Timing Report

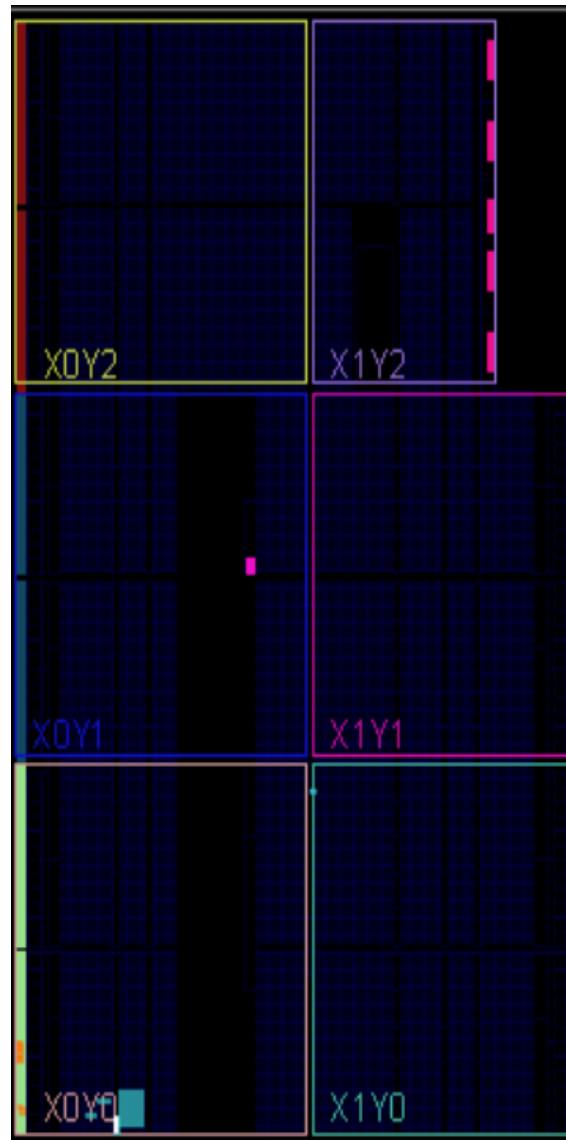


Critical path highlighted in the schematic

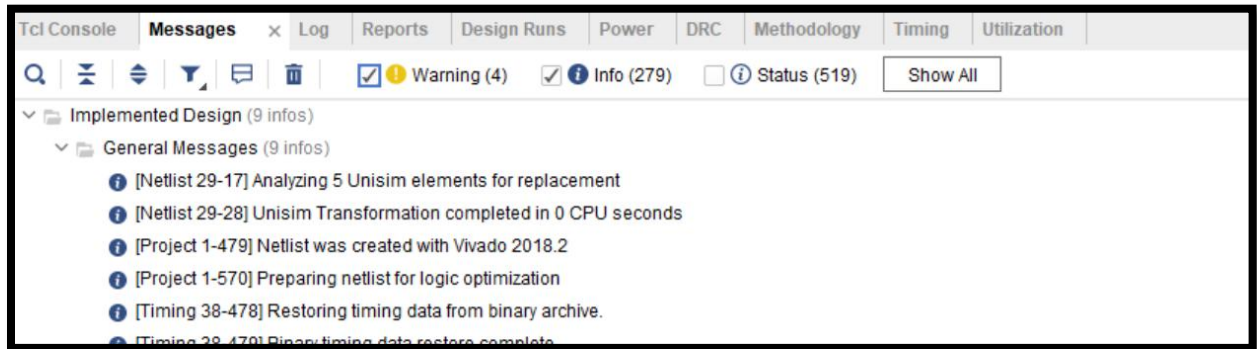


Implementation (seq encoding)

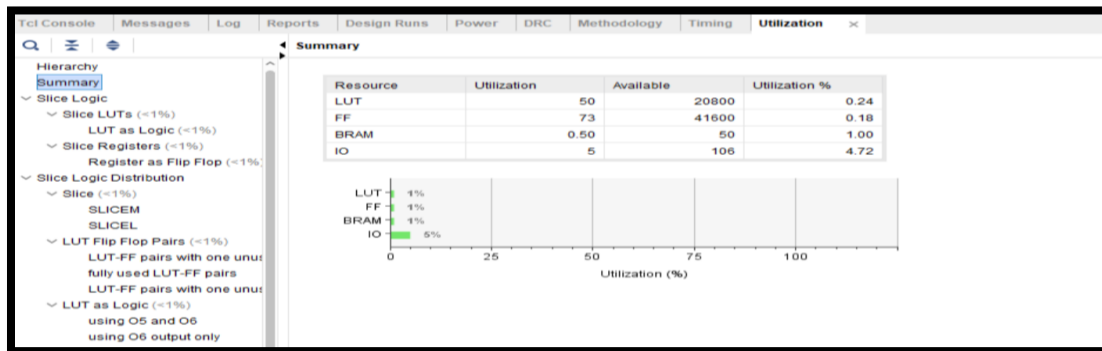
Device



Message



Utilization Report



Timing Report

The Timing Report window displays a design timing summary. The summary table shows the following data:

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.819 ns	Worst Hold Slack (WHS): 0.109 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 169	Total Number of Endpoints: 169	Total Number of Endpoints: 76

All user specified timing constraints are met.

Constrain File

Elaboration (gray encoding)

Message

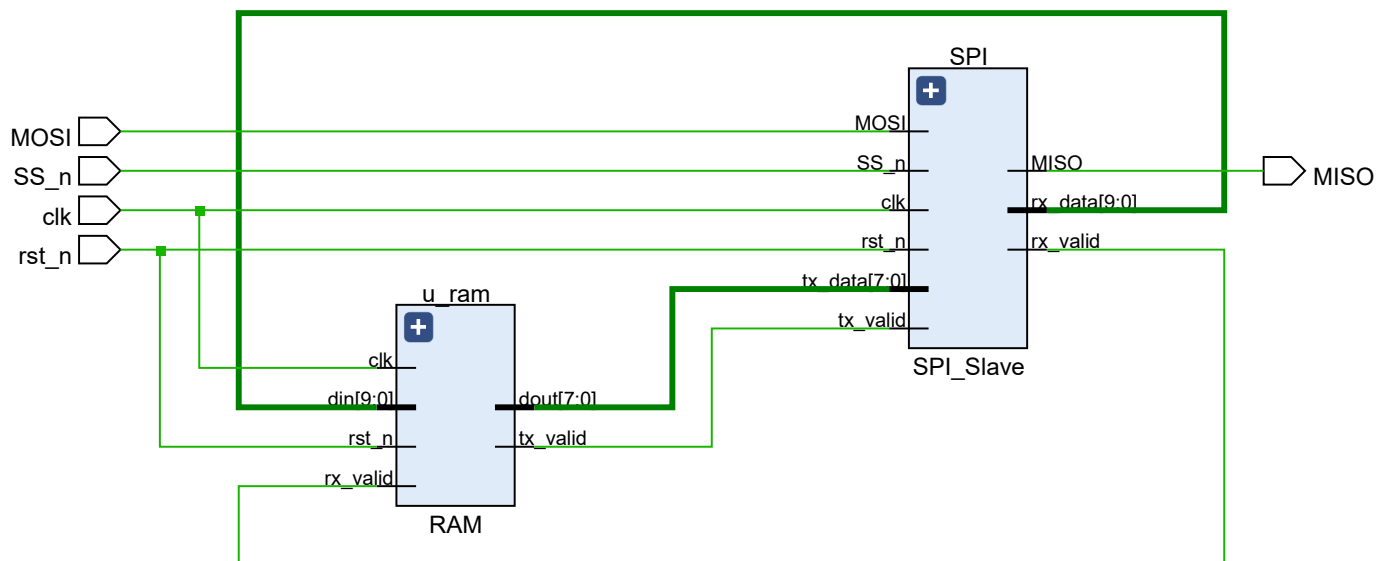
Elaborated Design (2 infos)

General Messages (2 infos)

[Project 1-570] Preparing netlist for logic optimization

[Opt 31-138] Pushed 0 inverter(s) to 0 load pin(s).

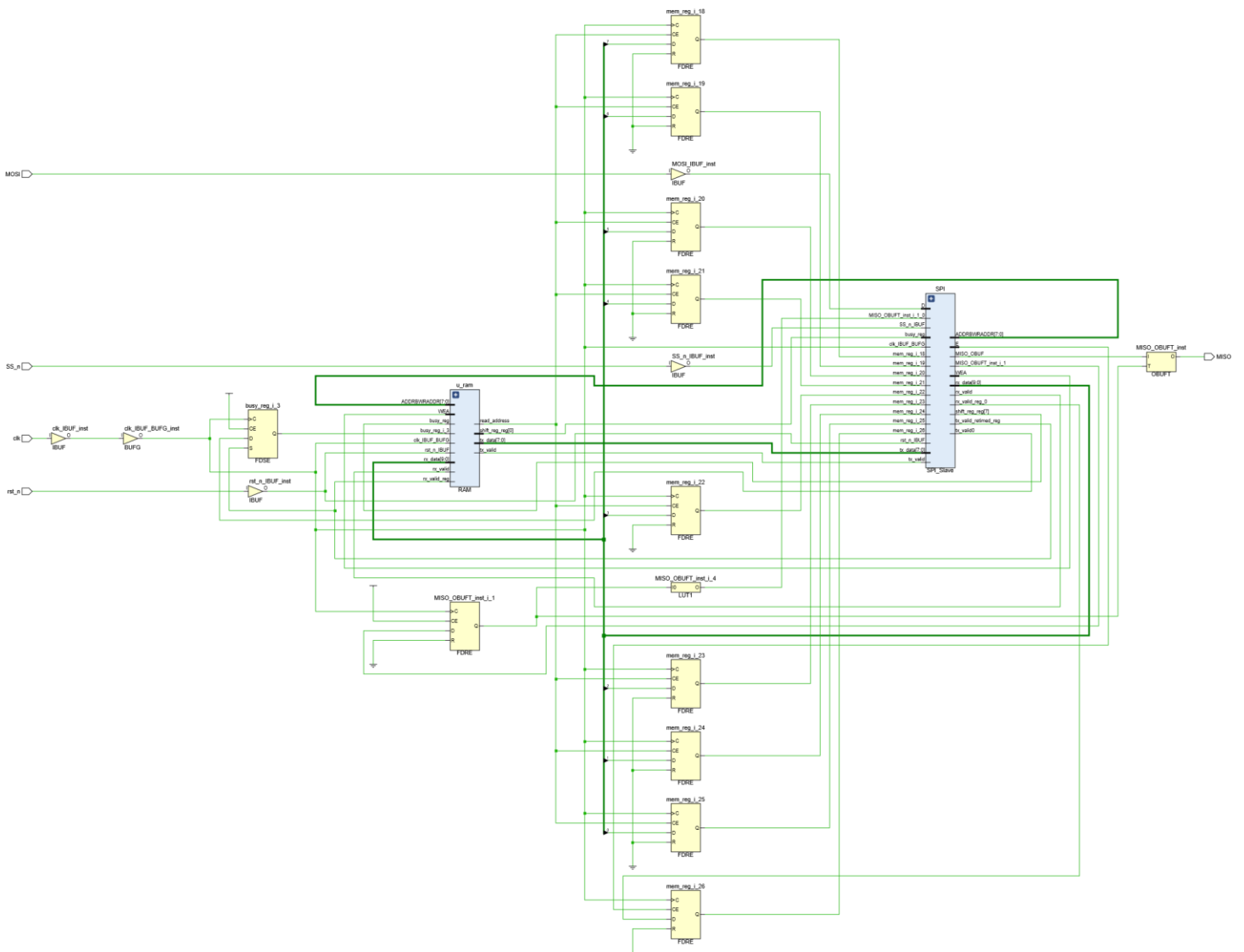
Schematic



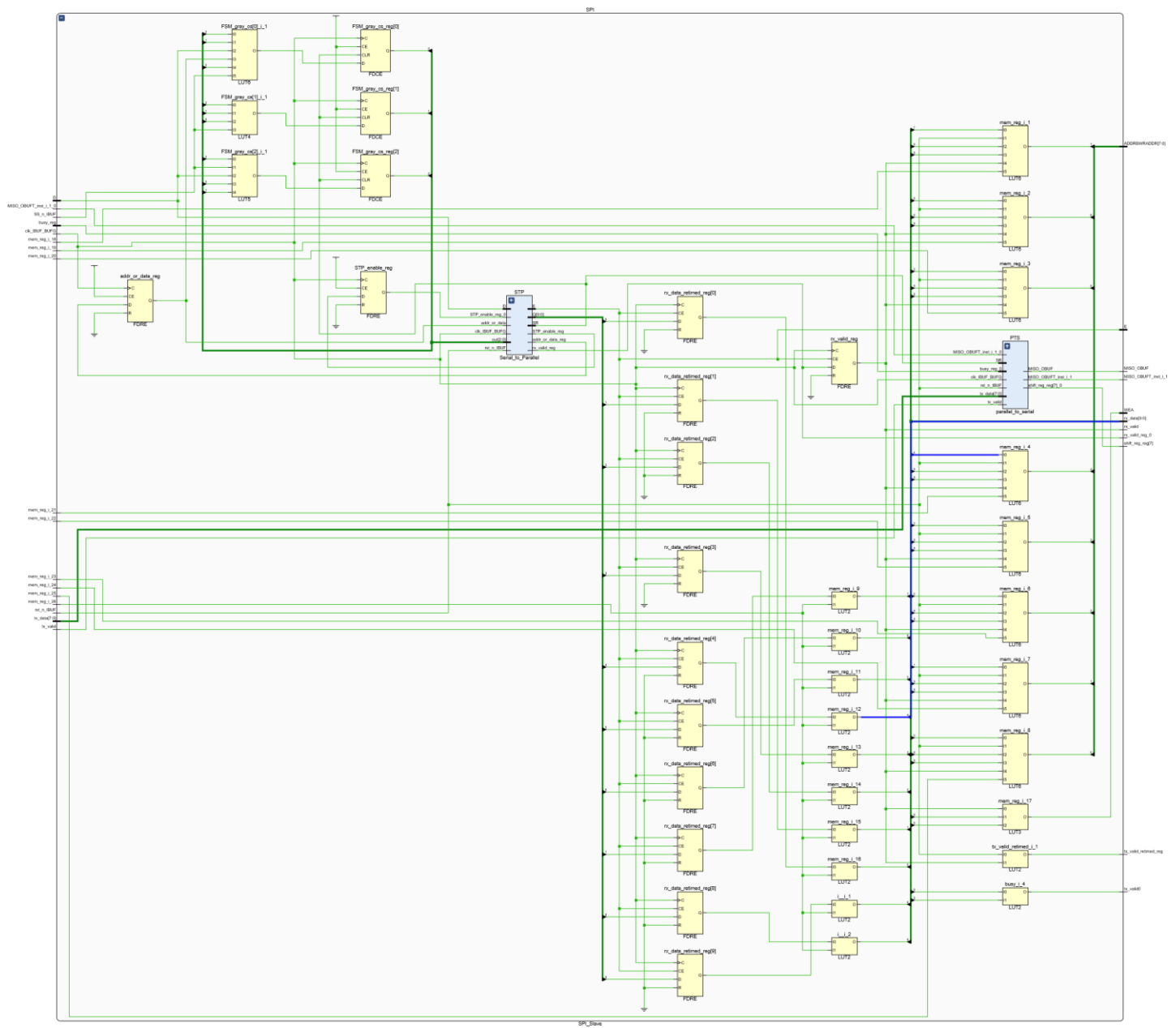
Synthesis (gray encoding)

State	Encoding
IDLE	5'b000
CHK_CMD	5'b001
WRITE	5'b011
READ_ADD	5'b010
READ_DATA	5'b110

Schematic



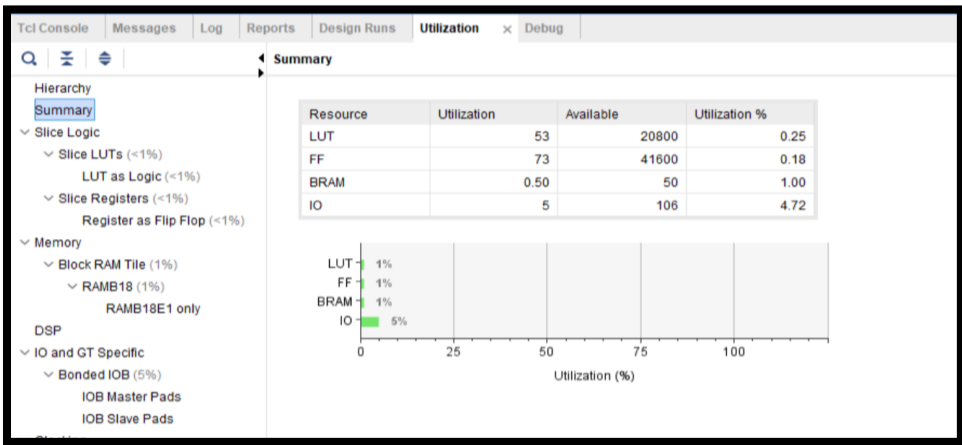
SPI_Slave Schematic



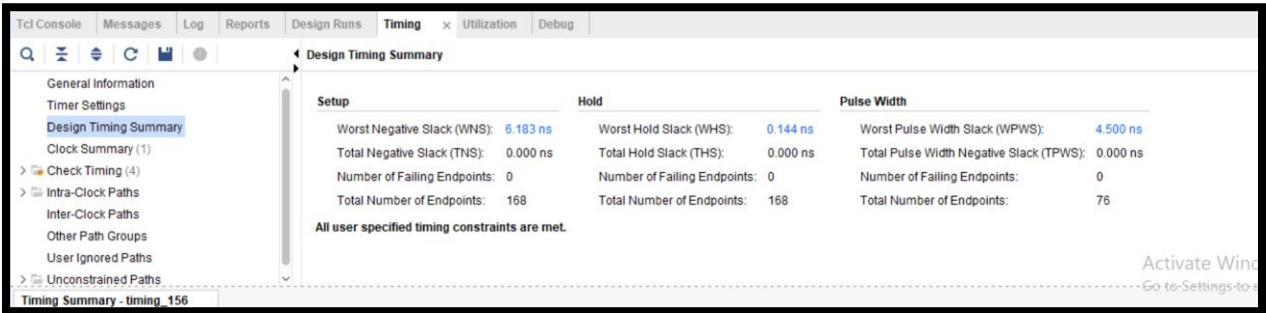
Message



Utilization Report

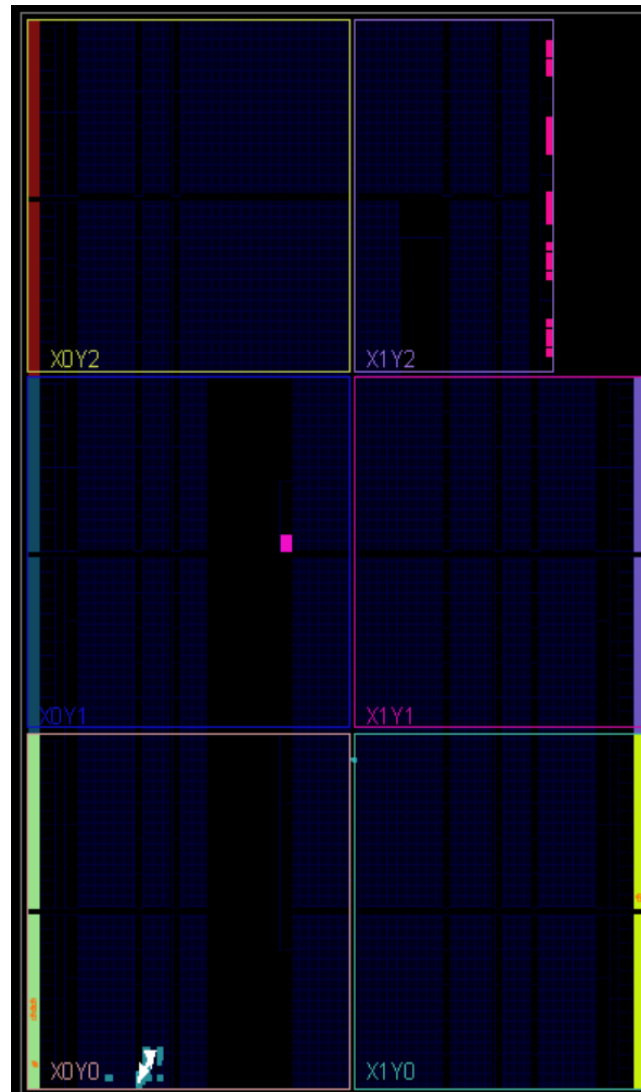


Timing Report

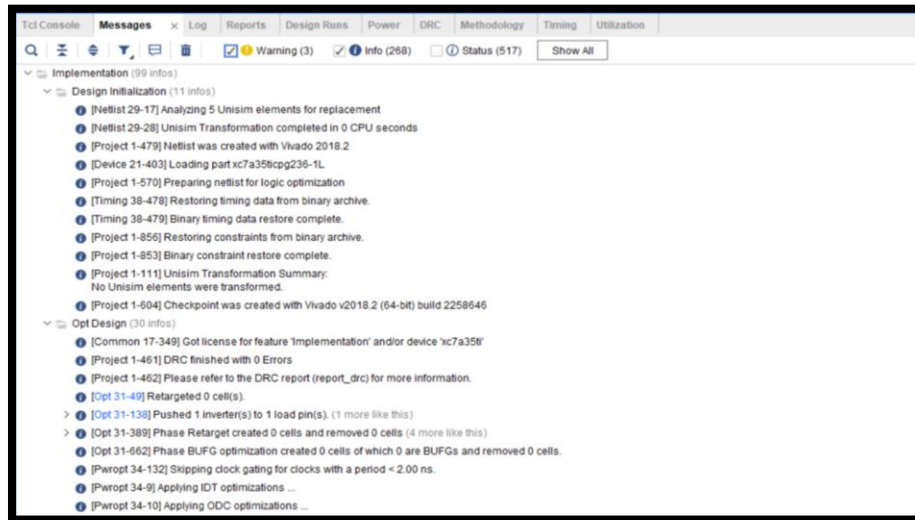


Implementation (gray encoding)

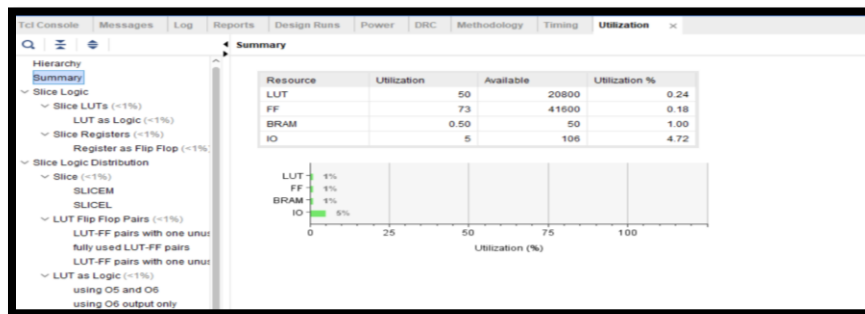
Device



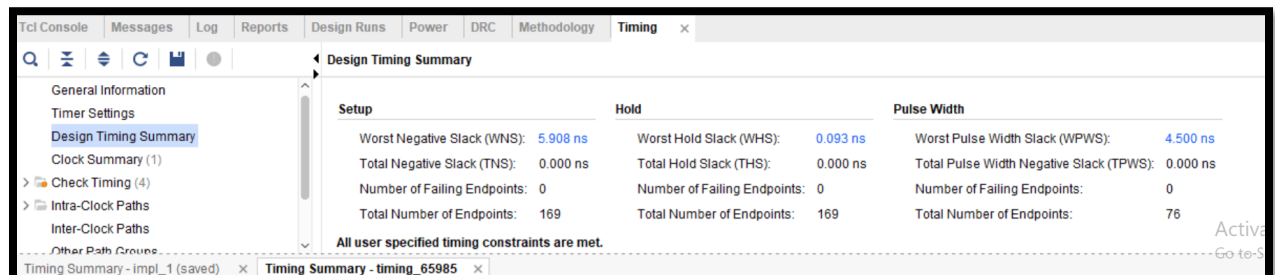
Message



Utilization Report



Timing Report

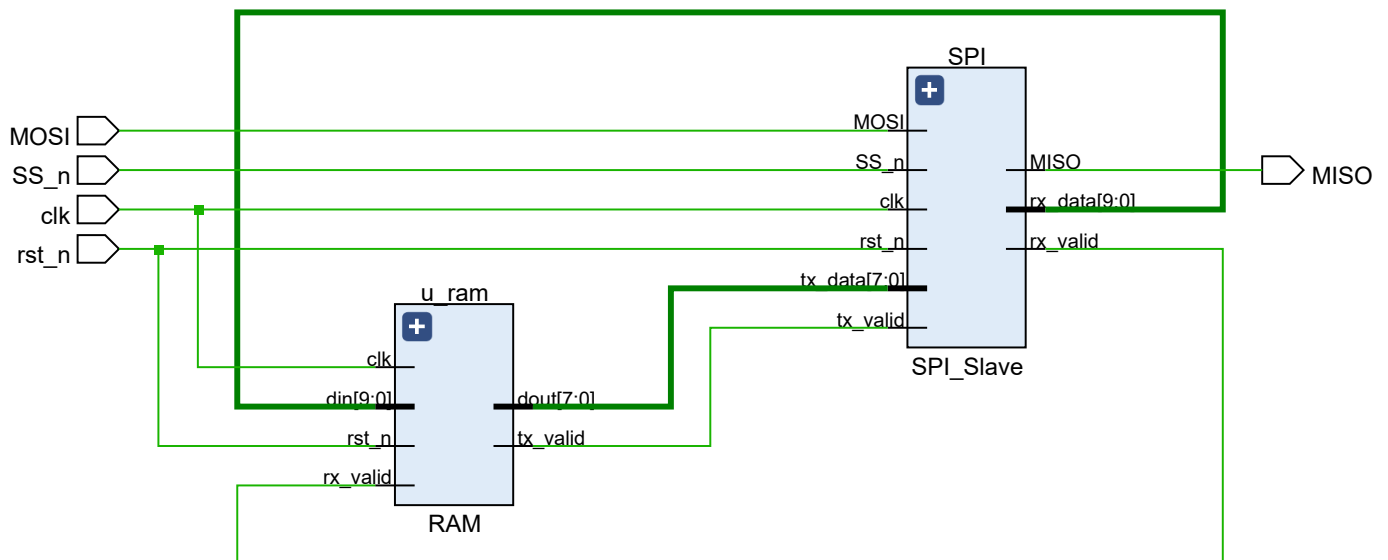


Elaboration (one_hot encoding)

Message



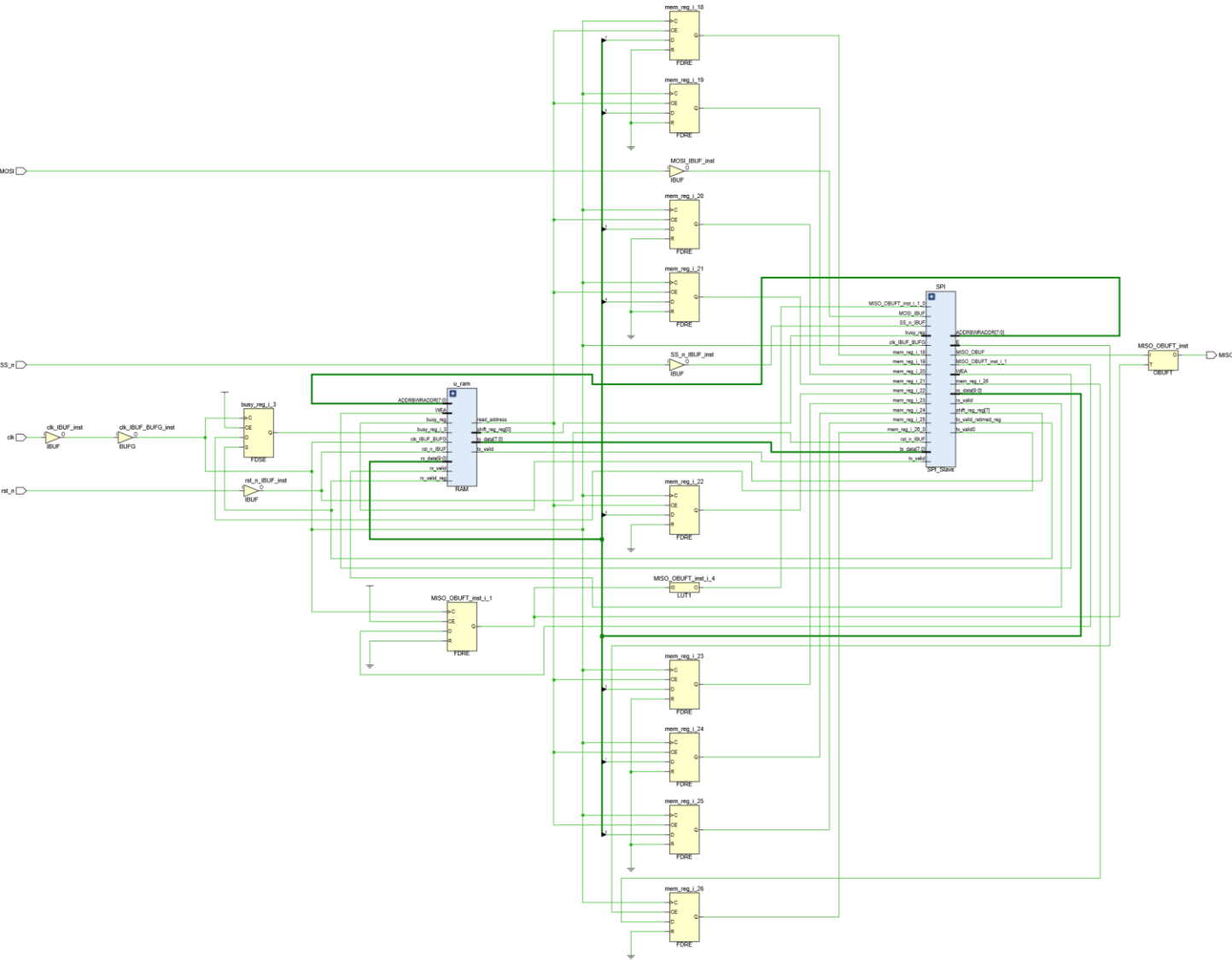
Schematic



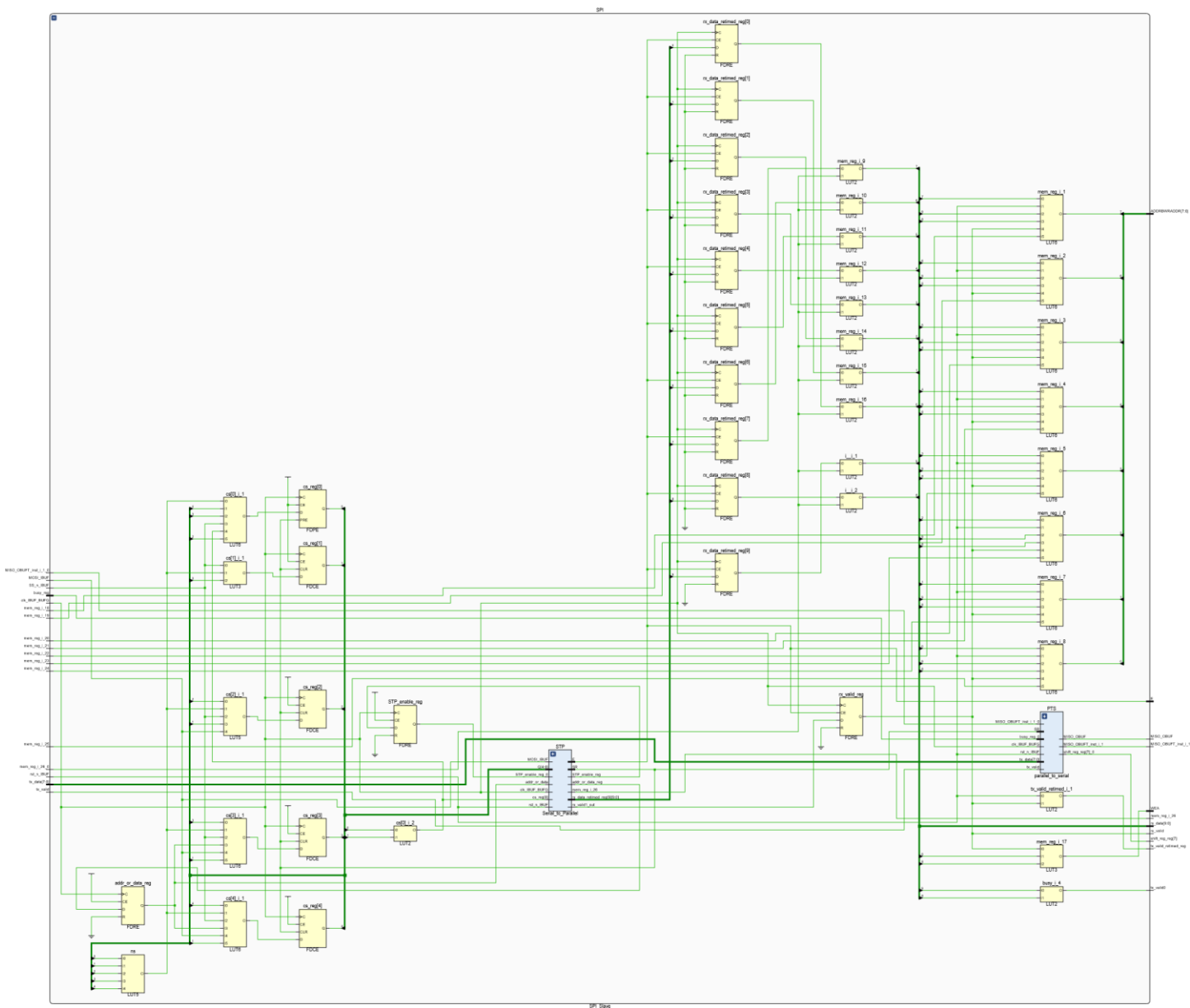
Synthesis (one_hot encoding)

State	Encoding
IDLE	5'b00001
CHK_CMD	5'b00010
WRITE	5'b00100
READ_ADD	5'b01000
READ_DATA	5'b10000

Schematic



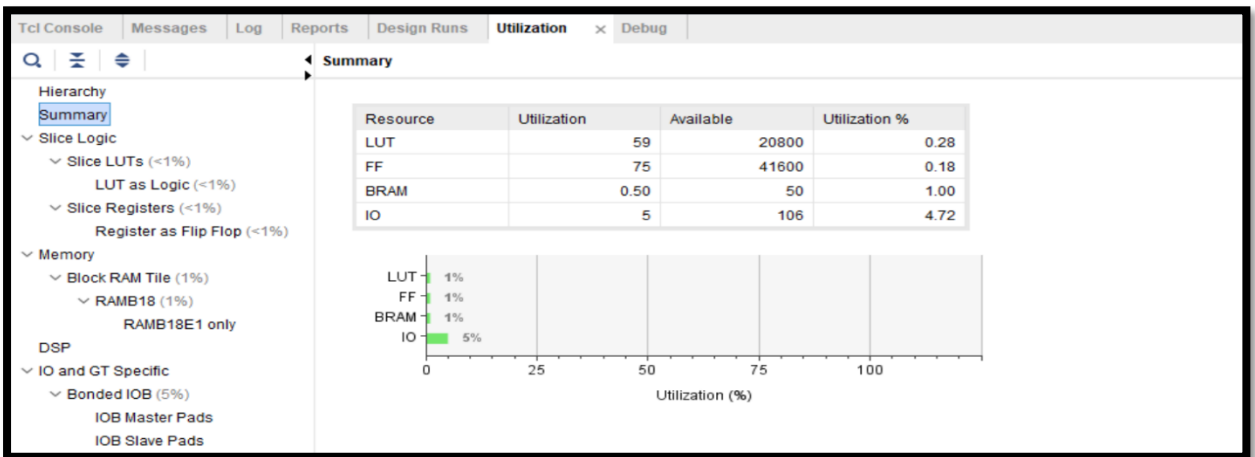
SPI_Slave Schematic



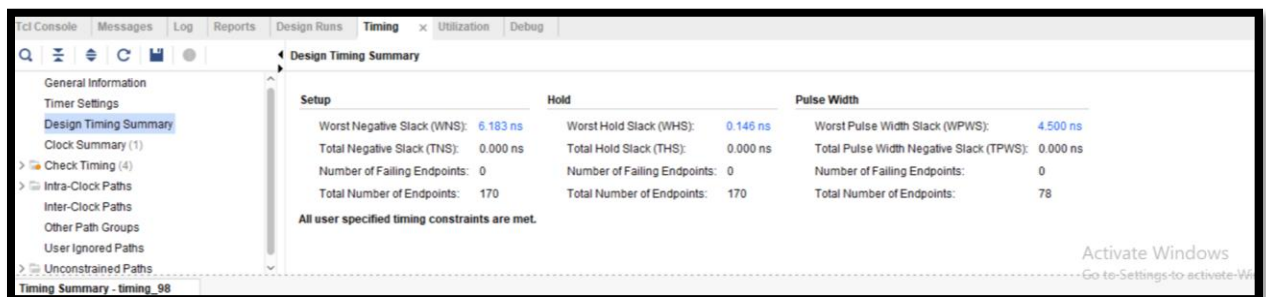
Message



Utilization Report

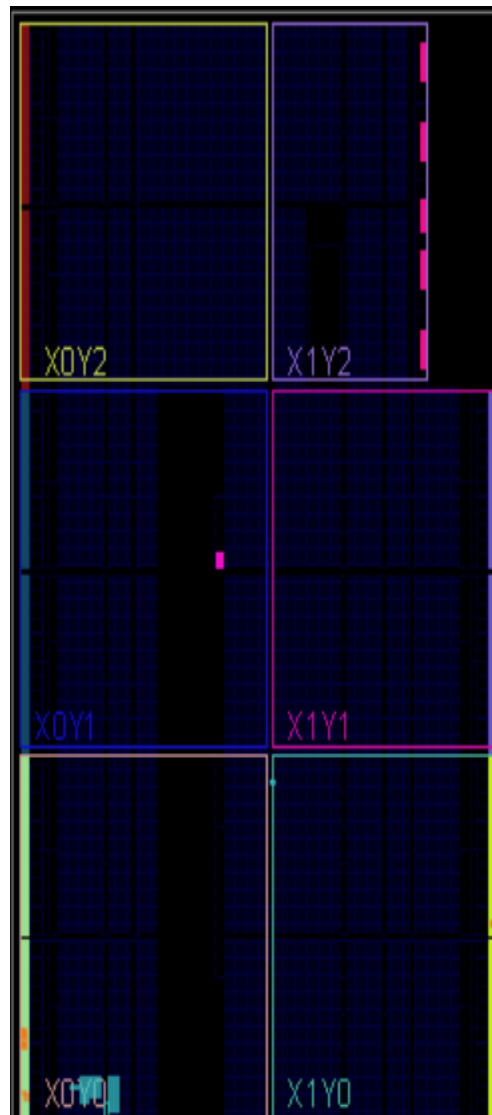


Timing Report

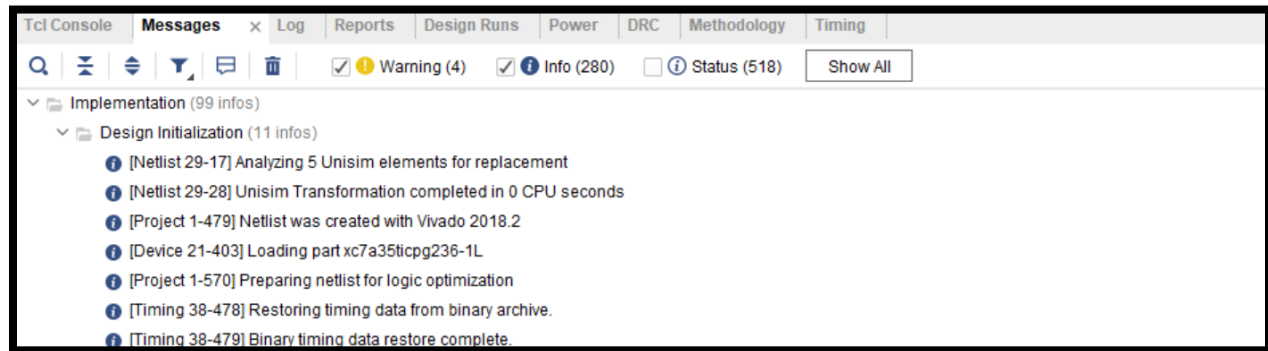


Implementation (one_hot encoding)

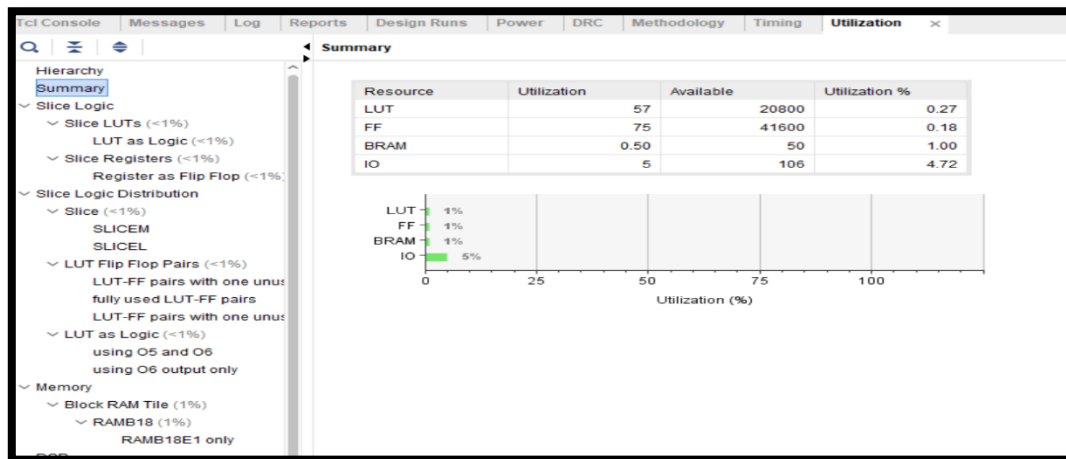
Device



Message



Utilization Report



Timing Report

The Timing Report window displays a design timing summary. The summary table is as follows:

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.959 ns	Worst Hold Slack (WHS): 0.104 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 171	Total Number of Endpoints: 171	Total Number of Endpoints: 78

All user specified timing constraints are met.

Debug Core



```
1  create_debug_core u_ila_0 ila
2  set_property ALL_PROBE_SAME_MU true [get_debug_cores u_ila_0]
3  set_property ALL_PROBE_SAME_MU_CNT 1 [get_debug_cores u_ila_0]
4  set_property C_ADV_TRIGGER false [get_debug_cores u_ila_0]
5  set_property C_DATA_DEPTH 1024 [get_debug_cores u_ila_0]
6  set_property C_EN_STRG_QUAL false [get_debug_cores u_ila_0]
7  set_property C_INPUT_PIPE_STAGES 0 [get_debug_cores u_ila_0]
8  set_property C_TRIGIN_EN false [get_debug_cores u_ila_0]
9  set_property C_TRIGOUT_EN false [get_debug_cores u_ila_0]
10 set_property port_width 1 [get_debug_ports u_ila_0/clock]
11 connect_debug_port u_ila_0/clock [get_nets [list clock_IBUF_BUFG]]
12 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe0]
13 set_property port_width 1 [get_debug_ports u_ila_0/probe0]
14 connect_debug_port u_ila_0/probe0 [get_nets [list clock_IBUF]]
15 create_debug_port u_ila_0 probe
16 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe1]
17 set_property port_width 1 [get_debug_ports u_ila_0/probe1]
18 connect_debug_port u_ila_0/probe1 [get_nets [list MISO_OBUF]]
19 create_debug_port u_ila_0 probe
20 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe2]
21 set_property port_width 1 [get_debug_ports u_ila_0/probe2]
22 connect_debug_port u_ila_0/probe2 [get_nets [list MOSI_IBUF]]
23 create_debug_port u_ila_0 probe
24 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe3]
25 set_property port_width 1 [get_debug_ports u_ila_0/probe3]
26 connect_debug_port u_ila_0/probe3 [get_nets [list rst_n_IBUF]]
27 create_debug_port u_ila_0 probe
28 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe4]
29 set_property port_width 1 [get_debug_ports u_ila_0/probe4]
30 connect_debug_port u_ila_0/probe4 [get_nets [list SS_n_IBUF]]
31 set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
32 set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
33 set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
34 connect_debug_port dbg_hub/clock [get_nets clock_IBUF_BUFG]
35
```